

Tutorial React: Construindo o Clone do Agar.io

Este guia passo a passo ensinará como construir uma versão funcional e otimizada do Agar.io baseada em componentes React. Vamos migrar a estrutura estática fornecida de HTML/CSS para o modelo de estados declarativos do React, integrando o motor do HTML5 Canvas com Hooks modernos (useState, useEffect, useRef).

O que vamos construir?

Abaixo está o código final unificado que você utilizará como base. Ele traz a separação exata proposta na documentação do React: Gerenciamento de Ciclo de Vida, Interface do Usuário (UI) controlada por estados e renderização gráfica desacoplada para garantir 60 quadros por segundo.

JavaScript

// App.jsx

```
import React, { useState, useEffect, useRef } from 'react';
```

```
// Constantes Globais de Configuração da Arena Física
```

```
const MAP_WIDTH = 3000;
```

```
const MAP_HEIGHT = 3000;
```

```
const MAX_FOODS = 600;
```

```
const TOTAL_BOTS = 25;
```

```
const COLORS = ['#ff1744', '#00e676', '#2979ff', '#ffea00', '#d500f9', '#ff9100',  
                '#00e5ff', '#ff4081'];
```

```
const BOT_NAMES = ['Poli', 'Alpha', 'WunWun', 'SplitMe', 'Blob', 'Doge', 'Sirius', 'Zeus',  
                  'Titan', 'Apex', 'Shadow'];
```

```
export default function AgarIoClone() {
```

```
  // 1. Estados Declarativos da Interface do Usuário (UI)
```

```
  const [gameState, setGameState] = useState('MENU'); // MENU | PLAYING
```

```
  const [playerName, setPlayerName] = useState('Célula');
```

```
  const [massScore, setMassScore] = useState(20);
```

```
  const [leaderboard, setLeaderboard] = useState([]);
```

```
// 2. Referências Mutáveis Permanentes (Evitam re-renderizações desnecessárias no React)
```

```
const canvasRef = useRef(null);
```

```
const worldRef = useRef({
```

```
  player: null,
```

```
  foods: [],
```

```
  bots: [],
```

```
  mouse: { x: 0, y: 0 },
```

```
  camera: { x: 0, y: 0 }
```

```
});
```

```
const animationFrameId = useRef(null);
```

```
// Instanciadores de Entidades do Ambiente Lógico
```

```
const createFood = () => ({
```

```
  x: Math.random() * MAP_WIDTH,
```

```
  y: Math.random() * MAP_HEIGHT,
```

```
  radius: 5,
```

```
  color: COLORS[Math.floor(Math.random() * COLORS.length)]
```

```
});
```

```
const createBot = () => ({
```

```
  id: Math.random(),
```

```
  x: Math.random() * MAP_WIDTH,
```

```
  y: Math.random() * MAP_HEIGHT,
```

```
  radius: Math.floor(Math.random() * 25) + 15,
```

```
  color: COLORS[Math.floor(Math.random() * COLORS.length)],
```

```
  name: BOT_NAMES[Math.floor(Math.random() * BOT_NAMES.length)],
```

```
  targetX: Math.random() * MAP_WIDTH,
```

```

    targetY: Math.random() * MAP_HEIGHT,
    decisionTime: 0
  });

// Inicializador de uma Nova Partida
const initGameWorld = () => {
  const foodsArray = Array.from({ length: MAX_FOODS }, createFood);
  const botsArray = Array.from({ length: TOTAL_BOTS }, createBot);

  worldRef.current = {
    player: {
      x: MAP_WIDTH / 2,
      y: MAP_HEIGHT / 2,
      radius: 20,
      color: '#00e5ff',
      name: playerName.trim() || 'Célula',
      dead: false
    },
    foods: foodsArray,
    bots: botsArray,
    mouse: { x: 0, y: 0 },
    camera: { x: 0, y: 0 }
  };
};

const getSpeed = (radius) => Math.max(1.1, 6.5 - (radius * 0.022));

// Inteligência Artificial Estocástica dos Bots

```

```

const processBotAI = (bot, player, bots, foods) => {

  bot.decisionTime--;

  if (bot.decisionTime <= 0) {

    bot.decisionTime = Math.floor(Math.random() * 50) + 30;


    // Detector de Predadores Próximos (Fuga)

    let threat = null;

    if (!player.dead && player.radius > bot.radius * 1.1 && Math.hypot(player.x - bot.x,
player.y - bot.y) < 280) {

      threat = player;

    }


    if (threat) {

      bot.targetX = Math.max(0, Math.min(MAP_WIDTH, bot.x + (bot.x - threat.x) * 2));

      bot.targetY = Math.max(0, Math.min(MAP_HEIGHT, bot.y + (bot.y - threat.y) * 2));

      return;

    }


    // Caça a Organismos Menores

    if (!player.dead && bot.radius > player.radius * 1.1 && Math.hypot(player.x - bot.x,
player.y - bot.y) < 220) {

      bot.targetX = player.x;

      bot.targetY = player.y;

      return;

    }


    // Forrageamento Padrão de Nutrientes

    if (foods.length > 0) {

      const sample = foods[Math.floor(Math.random() * foods.length)];

```

```
    bot.targetX = sample.x;
    bot.targetY = sample.y;
  }
}

// Movimentação Linear do Bot
const dx = bot.targetX - bot.x;
const dy = bot.targetY - bot.y;
const dist = Math.hypot(dx, dy);
if (dist > 5) {
  const speed = getSpeed(bot.radius);
  bot.x += (dx / dist) * speed;
  bot.y += (dy / dist) * speed;
}
};
```

```
// 3. Efeito Principal de Controle do Loop de Renderização e Eventos Globais
useEffect(() => {
  if (gameState !== 'PLAYING') return;

  const canvas = canvasRef.current;
  const ctx = canvas.getContext('2d');

  const handleResize = () => {
    canvas.width = window.innerWidth;
    canvas.height = window.innerHeight;
  };

  window.addEventListener('resize', handleResize);
```

```
handleResize();
```

```
const handleMouseMove = (e) => {  
  worldRef.current.mouse = { x: e.clientX, y: e.clientY };  
};  
window.addEventListener('mousemove', handleMouseMove);
```

```
// Loop do Motor de Física (60 FPS via RequestAnimationFrame)
```

```
const renderLoop = () => {  
  const world = worldRef.current;  
  if (!world || !world.player) return;  
  
  const { player, mouse, camera, foods, bots } = world;
```

```
// Cinemática do Jogador
```

```
if (!player.dead) {  
  const worldMouseX = mouse.x + camera.x;  
  const worldMouseY = mouse.y + camera.y;  
  const dx = worldMouseX - player.x;  
  const dy = worldMouseY - player.y;  
  const dist = Math.hypot(dx, dy);
```

```
  if (dist > 5) {  
    const speed = getSpeed(player.radius);  
    player.x += (dx / dist) * speed;  
    player.y += (dy / dist) * speed;  
  }
```

```
    player.x = Math.max(player.radius, Math.min(MAP_WIDTH - player.radius,
player.x));
```

```
    player.y = Math.max(player.radius, Math.min(MAP_HEIGHT - player.radius,
player.y));
```

```
    setMassScore(Math.floor(player.radius));
```

```
    // Centralização do foco da câmera
```

```
    camera.x = player.x - canvas.width / 2;
```

```
    camera.y = player.y - canvas.height / 2;
```

```
}
```

```
// Execução da IA dos Bots
```

```
bots.forEach(bot => {
```

```
    processBotAI(bot, player, bots, foods);
```

```
    bot.x = Math.max(bot.radius, Math.min(MAP_WIDTH - bot.radius, bot.x));
```

```
    bot.y = Math.max(bot.radius, Math.min(MAP_HEIGHT - bot.radius, bot.y));
```

```
});
```

```
// Resolução de Colisões de Nutrientes (Consumo)
```

```
world.foods = foods.filter(food => {
```

```
    if (!player.dead && Math.hypot(food.x - player.x, food.y - player.y) < player.radius) {
```

```
        player.radius += 0.25;
```

```
        return false;
```

```
    }
```

```
for (let bot of bots) {
```

```
    if (Math.hypot(food.x - bot.x, food.y - bot.y) < bot.radius) {
```

```
        bot.radius += 0.25;
```

```
        return false;
```

```

    }
}
return true;
});
while (world.foods.length < MAX_FOODS) world.foods.push(createFood());

```

// Lógica de Combate e Absorção de Células Oponentes

```

for (let i = 0; i < bots.length; i++) {
  for (let j = i + 1; j < bots.length; j++) {
    const b1 = bots[i]; const b2 = bots[j];
    const dist = Math.hypot(b1.x - b2.x, b1.y - b2.y);
    if (dist < b1.radius && b1.radius > b2.radius * 1.1) {
      b1.radius += b2.radius * 0.35; bots.splice(j, 1); j--;
    } else if (dist < b2.radius && b2.radius > b1.radius * 1.1) {
      b2.radius += b1.radius * 0.35; bots.splice(i, 1); i--; break;
    }
  }
}
}

```

```

if (!player.dead) {
  world.bots = bots.filter(bot => {
    const dist = Math.hypot(player.x - bot.x, player.y - bot.y);
    if (dist < player.radius && player.radius > bot.radius * 1.1) {
      player.radius += bot.radius * 0.35;
      return false;
    }
    if (dist < bot.radius && bot.radius > player.radius * 1.1) {
      player.dead = true;
    }
  });
}

```



```

    }

    return true;

  });
}

while (world.bots.length < TOTAL_BOTS) world.bots.push(createBot());

// Atualização Estruturada do Líderes (Leaderboard)

const allActive = [...world.bots];

if (!player.dead) allActive.push(player);

allActive.sort((a, b) => b.radius - a.radius);

setLeaderboard(allActive.slice(0, 10).map(c => ({ name: c.name, score:
Math.floor(c.radius) })));

// --- ETAPA DE DESENHO (CANVAS) ---

ctx.clearRect(0, 0, canvas.width, canvas.height);

ctx.save();

ctx.translate(-camera.x, -camera.y);

// Linhas limítrofes da arena

ctx.strokeStyle = '#ff1744'; ctx.lineWidth = 12;

ctx.strokeRect(0, 0, MAP_WIDTH, MAP_HEIGHT);

// Grade quadriculada de fundo (Grid)

ctx.strokeStyle = '#e2e2e2'; ctx.lineWidth = 1;

for (let x = 0; x < MAP_WIDTH; x += 50) {

  ctx.beginPath(); ctx.moveTo(x, 0); ctx.lineTo(x, MAP_HEIGHT); ctx.stroke();

}

for (let y = 0; y < MAP_HEIGHT; y += 50) {

```

```

    ctx.beginPath(); ctx.moveTo(0, y); ctx.lineTo(MAP_WIDTH, y); ctx.stroke();
}

// Renderizar Nutrientes
world.foods.forEach(f => {
    ctx.beginPath(); ctx.arc(f.x, f.y, f.radius, 0, Math.PI * 2);
    ctx.fillStyle = f.color; ctx.fill();
});

// Renderizar Bots
world.bots.forEach(b => {
    ctx.beginPath(); ctx.arc(b.x, b.y, b.radius, 0, Math.PI * 2);
    ctx.fillStyle = b.color; ctx.fill(); ctx.lineWidth = 2.5; ctx.strokeStyle = '#424242';
    ctx.stroke();
    ctx.fillStyle = 'white'; ctx.font = `bold ${Math.max(11, b.radius * 0.38)}px Arial`;
    ctx.textAlign = 'center'; ctx.textBaseline = 'middle'; ctx.fillText(b.name, b.x, b.y);
});

// Renderizar Célula do Jogador Humano
if (!player.dead) {
    ctx.beginPath(); ctx.arc(player.x, player.y, player.radius, 0, Math.PI * 2);
    ctx.fillStyle = player.color; ctx.fill(); ctx.lineWidth = 4; ctx.strokeStyle = '#0097a7';
    ctx.stroke();
    ctx.fillStyle = 'white'; ctx.font = `bold ${Math.max(11, player.radius * 0.38)}px
    Arial`;
    ctx.textAlign = 'center'; ctx.textBaseline = 'middle'; ctx.fillText(player.name,
    player.x, player.y);
}

```

```
ctx.restore();

animationFrameId.current = requestAnimationFrame(renderLoop);

};
```

```
animationFrameId.current = requestAnimationFrame(renderLoop);
```

```
// Retorno de limpeza completa (Garbage Collector)
```

```
return () => {

  cancelAnimationFrame(animationFrameId.current);

  window.removeEventListener('resize', handleResize);

  window.removeEventListener('mousemove', handleMouseMove);

};

}, [gameState]);
```

```
const startGame = () => {

  initGameWorld();

  setGameState('PLAYING');

};
```

```
return (

  <div style={{ width: '100vw', height: '100vh', overflow: 'hidden', position: 'relative',
margin: 0, padding: 0 }}>

    {/* MENU INICIAL CONTROLANDO O ESTADO DO REACTION */}

    {gameState === 'MENU' && (

      <div style={{ position: 'fixed', top: 0, left: 0, width: '100%', height: '100%',
backgroundColor: 'rgba(28, 28, 28, 0.8)', display: 'flex', justifyContent: 'center',
alignItems: 'center', zIndex: 10 }}>

        <div style={{ background: '#ffffff', padding: '40px', borderRadius: '10px',
boxShadow: '0px 10px 30px rgba(0,0,0,0.3)', textAlign: 'center', minWidth: '320px' }}>
```

```

    <h1 style={{ fontSize: '3.5rem', color: '#333', marginBottom: '25px', fontWeight:
900 }}>AGAR.IO</h1>

    <input

      type="text"

      value={playerName}

      onChange={(e) => setPlayerName(e.target.value)}

      style={{ width: '100%', padding: '12px', marginBottom: '15px', border: '2px solid
#ddd', borderRadius: '5px', fontSize: '1.1rem', outline: 'none', textAlign: 'center' }}

      placeholder="Seu Apelido"

      maxLength="12"

    />

    <button onClick={startGame} style={{ width: '100%', padding: '14px',
backgroundColor: '#00e676', color: white, border: 'none', borderRadius: '5px', fontSize:
'1.3rem', fontWeight: 'bold', cursor: 'pointer' }}>

      Iniciar Jogo

    </button>

  </div>

</div>

)}

{/* RENDERIZAÇÃO CONDICIONAL DA HUD, LEADERBOARD E ARENA */}

{gameState === 'PLAYING' && (

  <>

    <div style={{ position: 'absolute', top: '20px', left: '20px', zIndex: 5,
backgroundColor: 'rgba(0, 0, 0, 0.6)', padding: '15px', borderRadius: '6px', color: 'white'
}}>

      <div style={{ fontSize: '1.3rem', fontWeight: 'bold', marginBottom: '8px'
}}>Massa: {massScore}</div>

      <button onClick={() => setGameState('MENU')} style={{ backgroundColor:
'ff1744', color: white, border: 'none', padding: '8px 15px', borderRadius: '4px', cursor:
'pointer', fontWeight: 'bold', width: '100%' }}>

```

```

    Sair

  </button>

</div>

<div style={{ position: 'absolute', top: '20px', right: '20px', zIndex: 5,
backgroundColor: 'rgba(255, 255, 255, 0.4)', color: '#333', padding: '15px',
borderRadius: '5px', minWidth: '180px' }}>

  <h3 style={{ textAlign: 'center', marginBottom: '8px', fontSize: '1.2rem',
borderBottom: '1px solid rgba(0,0,0,0.2)', paddingBottom: '5px' }}>Líderes</h3>

  <ol style={{ paddingLeft: '25px', fontWeight: 'bold', fontSize: '0.95rem' }}>

    {leaderboard.map((item, idx) => (

      <li key={idx} style={{ marginBottom: '4px', color: item.name === playerName ?
'#00b0ff' : 'inherit' }}>

        {item.name}: {item.score}

      </li>

    ))}

  </ol>

</div>

<canvas ref={canvasRef} style={{ display: 'block', width: '100vw', height: '100vh' }}
/>

</>

  </div>

);

}

```

Parte 1: Fundamentos de Arquitetura e Estrutura de Estados

A migração de uma página web estática com lógica sequencial em JavaScript para um ecossistema declarativo em React exige uma mudança profunda de paradigma. Enquanto o código legado manipula diretamente elementos do DOM

(document.getElementById), o React utiliza o conceito de **Estados Reativos** para ditar o comportamento do documento.

1.1 Gerenciamento do Ciclo de Vida com o Hook useState

No código anterior, a transição entre telas dependia de adicionar ou remover classes CSS utilitárias como .escondido. No React, adotamos a **Renderização Condicional Baseada em Estados**.

O estado fundamental definido no nosso projeto é o gameState:

JavaScript

```
const [gameState, setGameState] = useState('MENU');
```

Esse estado age como o controlador mestre da aplicação. Se seu valor for 'MENU', o React remove completamente os nós da HUD e do Canvas da árvore do DOM virtual, injetando apenas o formulário de login. Quando o botão "Iniciar Jogo" altera o estado para 'PLAYING', o React reconstrói dinamicamente a interface na tela.

1.2 O Papel das Referências com useRef

Ao desenvolver jogos com altas taxas de atualização (60 FPS), o ciclo tradicional de renderização do React torna-se um gargalo de desempenho. Se armazenássemos as posições cartesianas de 600 comidas e 25 bots dentro de estados tradicionais (useState), o React tentaria disparar 60 atualizações por segundo para cada elemento, travando imediatamente o navegador.

Para mitigar esse problema, introduzimos o Hook useRef:

JavaScript

```
const worldRef = useRef({ player: null, foods: [], bots: [] });
```

A referência cria um contêiner mutável persistente que dura todo o ciclo de vida do componente. Modificar propriedades dentro de worldRef.current altera os valores na memória do computador instantaneamente, mas **não** força o React a re-renderizar o HTML, deixando o processamento matemático livre de amarras estruturais.

Parte 2: O Motor de Física e Geometria Vetorial

O motor físico do jogo opera de forma agnóstica às mecânicas de renderização do React. Ele processa vetores no espaço bidimensional utilizando fórmulas matemáticas aplicadas a círculos perfeitos.

2.1 Teorema de Pitágoras e Distância Euclidiana

Para verificar qualquer interação no jogo (se o jogador comeu um nutriente ou se um bot engoliu outro organismo), aplicamos a fórmula da distância euclidiana.

Lematicamente, duas circunferências entram em colisão quando a distância entre seus eixos de centro (X, Y) é menor que a soma de seus respectivos raios (R).

A equação utilizada pelo interpretador JavaScript calcula a hipotenusa dos eixos cartesianos:

$$\text{Distância} = \sqrt{(X_1 - X_2)^2 + (Y_1 - Y_2)^2}$$

No código, essa lógica é traduzida usando o método de alta performance `Math.hypot`:

JavaScript

```
if (Math.hypot(food.x - player.x, food.y - player.y) < player.radius) {  
    player.radius += 0.25; // Estado de Crescimento Confirmado  
}
```

2.2 Física de Arrasto e Dinâmica de Velocidade

Para garantir o equilíbrio competitivo das partidas, as células maiores devem perder velocidade linear gradativamente, evitando que um organismo gigante domine o mapa sem chances de defesa para os novatos. Criamos uma função matemática de amortecimento baseada no raio da célula:

$$\text{Velocidade} = \max(1.1, 6.5 - (\text{raio} \times 0.022))$$

Essa equação dita que o limite de velocidade decai de forma linear até atingir o piso absoluto estável de 1.1 px por frame, balanceando o jogo.

Parte 3: Inteligência Artificial Estocástica (Bots)

Cada um dos 25 bots inseridos simula o comportamento de tomada de decisão de um jogador humano por meio de uma máquina de estados simplificada.

3.1 O Algoritmo de Prioridade Vetorial

O comportamento dos bots é avaliado de forma assíncrona por meio de um temporizador individual (`decisionTime`). Essa abordagem evita sobrecarregar o processador central da máquina. A estrutura de tomada de decisão segue uma árvore de prioridades rígida:

1. **Estado de Evasão (Medo):** O bot escaneia a arena. Se encontrar o jogador humano ou um oponente robô com raio 10% superior a uma distância menor que 280 pixels, ele inverte o vetor de seu alvo atual para correr na direção oposta ao agressor.
2. **Estado de Predação (Caça):** Caso o ambiente esteja seguro de grandes ameaças, o bot busca células menores dentro de um raio de 220 pixels. Se

encontrar, suas coordenadas de destino (targetX, targetY) são travadas no centro da presa.

3. **Estado de Coleta (Alimentação):** Na ausência de ameaças e presas, o bot seleciona um índice aleatório dentro do vetor global de nutrientes ativos para manter seu ritmo de crescimento estável.

Parte 4: Projeção Gráfica e Matriz de Câmera

A área de desenho utilizável por um monitor padrão restringe-se às dimensões físicas da janela aberta no navegador. Como nossa arena lógica possui um tamanho imutável de 3000×3000 pixels, seria impossível acompanhar a partida sem mover o ponto de vista do jogador.

4.1 Translação de Coordenadas Globais para Locais

Para criar o efeito de câmera contínua que segue o organismo controlado pelo usuário, calculamos o deslocamento linear com base no centro do Canvas. A câmera armazena a discrepância matemática entre a posição absoluta do jogador no mapa e as dimensões da janela física do usuário:

$$X_{\text{câmera}} = X_{\text{jogador}} - \frac{\text{Largura da Tela}}{2}$$

$$Y_{\text{câmera}} = Y_{\text{jogador}} - \frac{\text{Altura da Tela}}{2}$$

Uma vez obtidos os valores de deslocamento, salvamos a matriz limpa do Canvas, aplicamos uma translação inversa equivalente antes de renderizar as imagens e limpamos os registros gráficos para o próximo quadro:

JavaScript

```
ctx.save();
```

```
ctx.translate(-camera.x, -camera.y);
```

```
// Desenho de todos os elementos usando coordenadas absolutas do mapa
```

```
ctx.restore();
```

Essa operação limpa e otimiza o processamento geométrico da GPU, eliminando travamentos e entregando uma experiência fluida para a aplicação.